Firstly, a single target phrase representation is created by using a pooling layer that takes the average over the columns of $\boldsymbol{F}^t$, as shown in (17).

$$\underset{d_f^t \times 1}{\boldsymbol{r}^t} = \frac{1}{n_f^t} \sum_{i=1}^{n_f^t} \underset{d_f^t \times 1}{\boldsymbol{f}_i^t} . \tag{17}$$

$\boldsymbol{r}^t$ is then used as a query to create a context vector out of the left and right contexts, separately. For example, for the left context, the key vectors $\boldsymbol{k}_1^l, \ldots, \boldsymbol{k}_{n_f^l}^l \in \mathbb{R}^{d_k^l}$ and value vectors $\boldsymbol{v}_1^l, \ldots, \boldsymbol{v}_{n_f^l}^l \in \mathbb{R}^{d_v^l}$ are extracted from the left context feature vectors $\boldsymbol{f}_1^l, \ldots, \boldsymbol{f}_{n_f^l}^l \in \mathbb{R}^{d_f^l}$, similarly as before, where d_k^l and d_v^l are the dimensions of the key and value vectors, respectively. Note that [36] proposes to use the original feature vectors as keys and values, meaning that the linear transformation consists of a multiplication by an identity matrix. Next, the scores are calculated using (18).

$$\underset{1 \times 1}{e_i^l} = \text{score}(\underset{d_f^t \times 1}{\boldsymbol{r}^t} , \underset{d_k^l \times 1}{\boldsymbol{k}_i^l}). \tag{18}$$

For the score function, [36] proposes to use an activated general score function [34] with a tanh activation function. The attention scores can be combined with an alignment function and the corresponding value vectors to produce the context vector $\boldsymbol{r}^l \in \mathbb{R}^{d_v^l}$. The alignment function used in [36] takes the form of a softmax function. An analogous procedure can be performed to obtain the representation of the right context, $\boldsymbol{r}^r$. These two context representations can then be used to create new representations of the target phrase, again, using attention. Firstly, the key vectors $\boldsymbol{k}_1^t, \ldots, \boldsymbol{k}_{n_f^t}^t \in \mathbb{R}^{d_k^t}$ and value vectors $\boldsymbol{v}_1^t, \ldots, \boldsymbol{v}_{n_f^t}^t \in \mathbb{R}^{d_v^t}$ are extracted from the target phrase feature vectors $\boldsymbol{f}_1^t, \ldots, \boldsymbol{f}_{n_f^t}^t \in \mathbb{R}^{d_f^t}$, similarly as before, using a linear transformation, where d_k^t and d_v^t are the dimensions of the key and value vectors, respectively. Note, again, that the original feature vectors as keys and values in [36]. The attention scores for the left-aware target representation are then calculated using (19).

$$\underset{1 \times 1}{e_i^{l_t}} = \text{score}(\underset{d_v^l \times 1}{\boldsymbol{r}^l} , \underset{d_k^t \times 1}{\boldsymbol{k}_i^t}). \tag{19}$$

The attention scores can be combined with an alignment function and the corresponding value vectors to produce the context vector $\boldsymbol{r}^{l_t} \in \mathbb{R}^{d_v^t}$. For this attention calculation, [34] proposes to use the same score and alignment functions as before. The right-aware target representation $\boldsymbol{r}^{r_t}$ can be calculated in a similar manner. Finally, to obtain the full representation vector $\boldsymbol{r}$ that is used to determine the classification, the vectors $\boldsymbol{r}^l$, $\boldsymbol{r}^r$, $\boldsymbol{r}^{l_t}$, and $\boldsymbol{r}^{r_t}$ are concatenated together, as shown in (20).

$$\underset{(d_v^l + d_v^r + d_v^t + d_v^t) \times 1}{\boldsymbol{r}} = \text{concat}(\underset{d_v^l \times 1}{\boldsymbol{r}^l} , \underset{d_v^r \times 1}{\boldsymbol{r}^r} , \underset{d_v^t \times 1}{\boldsymbol{r}^{l_t}} , \underset{d_v^t \times 1}{\boldsymbol{r}^{r_t}}). \tag{20}$$

To summarize, rotatory attention uses the target phrase to compute new representations for the left and right context using attention, and then uses these left and right representations to calculate new representations for the target phrase. The first step is designed to capture the words in the left and right contexts that are most important to the target phrase. The second step is there to capture the most important information in the actual target phrase itself. Essentially, the mechanism rotates attention between the target and the contexts to improve the representations.

There are many applications where combining information from different inputs into a single model can be highly beneficial. For example, in the field of medical data, there are often many different types of data available, such as various scans or documents, that can provide different types of information. In [37], a co-attention mechanism is used for automatic medical report generation to attend to both images and semantic tags simultaneously. Similarly, in [38], a co-attention model is proposed that combines general demographics features and patient medical history features to predict future health information. Additionally, an ablation study is used in [38] to show that the co-attention part of the model specifically improves performance. A field where multi-feature attention has been extensively explored is the domain of recommender systems. For example, in [39], a co-attention network is proposed that attends to both product reviews and the reviews a user has written. In [40], a model is proposed for video recommendation that attends to both user features and video features. Co-attention techniques have also been used in combination with graph networks for the purpose of, for example, reading comprehension across multiple documents [41] and fake news detection [42]. In comparison to co-attention, rotatory attention has typically been explored only in the field of sentiment analysis, which is most likely due to the specific structure of the data that is necessary to use this technique. An implementation of rotatory attention is proposed in [43] for sentiment analysis, where the mechanism is extended by repeating the attention rotation to iteratively further improve the representations.

3.1.2 Feature Levels

The previously discussed attention mechanisms process data at a single level. We refer to these attention techniques as **single-level attention** mechanisms. However, some data types can be analyzed and represented on multiple levels. For example, when analyzing documents, one can analyze the document at the sentence level, word level, or even the character level. When representations or embeddings of all these levels are available, one can exploit the extra levels of information. For example, one could choose to perform translation based on either just the characters, or just the words of the sentence. However, in [44], a technique named **attention-via-attention** is introduced that allows one to incorporate information from both the character, and the word levels. The idea is to predict the sentence translation character-by-character, while also incorporating information from a word-level attention module.

To begin with, a feature model (consisting of, for example, word embeddings and RNNs) is used to encode the input sentence into both a character-level feature matrix $\boldsymbol{F}^{(c)} \in \mathbb{R}^{d_f^{(c)} \times n_f^{(c)}}$, and a word-level feature matrix $\boldsymbol{F}^{(w)} \in \mathbb{R}^{d_f^{(w)} \times n_f^{(w)}}$, where $d_f^{(c)}$ and $n_f^{(c)}$ represent, respectively, the dimension of the embeddings of the characters, and the number of characters, while $d_f^{(w)}$ and $n_f^{(w)}$ represent the same but at the word level. It is crucial for this method that each level in the data can be represented or embedded. When attempting to predict a character in the translated

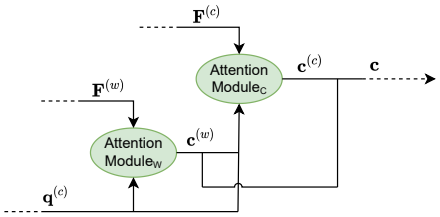

Fig. 7. An illustration of attention-via-attention.

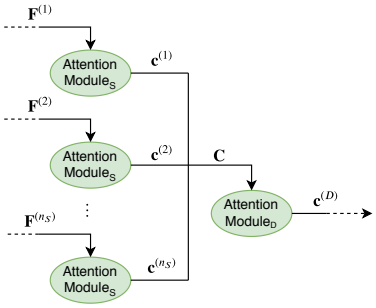

Fig. 8. An illustration of hierarchical attention.

sentence, a query $\boldsymbol{q}^{(c)} \in \mathbb{R}^{d_q}$ is created by the query model (like a character-level RNN), where d_q is the dimension of the query vectors. As illustrated in Fig. 7, the query is used to calculate attention on the word-level feature vectors $\boldsymbol{F}^{(w)}$. This generates the context vector $\boldsymbol{c}^{(w)} \in \mathbb{R}^{d_v^{(w)}}$, where $d_v^{(w)}$ represents the dimension of the value vectors for the word-level attention module. This context vector summarizes which words contain the most important information for predicting the next character. If we know which words are most important, then it becomes easier to identify which characters in the input sentence are most important. Thus, the next step is to attend to the character-level features in $\boldsymbol{F}^{(c)}$, with an additional query input: the word-level context vector $\boldsymbol{c}^{(w)}$. The actual query input for the attention model will therefore be the concatenation of the query $\boldsymbol{q}^{(c)}$ and the word context vector $\boldsymbol{c}^{(w)}$. The output of this character-level attention module is the context vector $\boldsymbol{c}^{(c)}$. The complete context output of the attention model is the concatenation of the word-level, and character-level context vectors.

The attention-via-attention technique uses representations for each level. However, accurate representations may not always be available for each level of the data, or it may be desirable to let the model create the representations during the process by building them from lower level representations. A technique referred to as **hierarchical attention** [5] can be used in this situation. Hierarchical attention is another technique that allows one to apply attention on different levels of the data. Yet, the exact mechanisms work quite differently compared to attention-via-attention. The idea is to start at the lowest level, and then create representations, or summaries, of the next level using attention. This process is repeated till the highest level is reached. To make this a little clearer, suppose one attempts to create a model for document classification, similarly to the implementation from [5]. We analyze a document containing n_S sentences, with the sth sentence containing n_s words, for $s = 1, \ldots, n_S$. One could use attention based on just the collection of words to classify the document. However, a significant amount of important context is then left out of the analysis, since the model will consider all words as a single long sentence, and will therefore not consider the context within the separate sentences. Instead, one can use the hierarchical structure of a document (words form sentences, and sentences form the document).

Fig. 8 illustrates the structure of hierarchical attention. For each sentence in the document, a sentence representation $\boldsymbol{c}^{(s)} \in \mathbb{R}^{d_v^{(S)}}$ is produced, for $s = 1, \ldots, n_S$, where $d_v^{(S)}$ is the dimension of the value vectors used in the attention model for sentence representations (Attention Module$_\text{S}$ in Fig. 8). The representation is a context vector from an attention module that essentially summarizes the sentence. Each sentence is first put through a feature model to extract the feature matrix $\boldsymbol{F}^{(s)} \in \mathbb{R}^{d_f^{(S)} \times n_s}$, for $s = 1, \ldots, n_S$, where $d_f^{(S)}$ represents the dimension of the feature vector for each word, and n_s represents the amount of words in sentence s. For extra clarification, the columns of $\boldsymbol{F}^{(s)}$ are feature vectors that correspond to the words in sentence s. As shown in Fig. 8, each feature matrix $\boldsymbol{F}^{(s)}$ is used as input for an attention model, which produces the context vector $\boldsymbol{c}^{(s)}$, for each $s = 1, \ldots, n_S$. No queries are used in this step, so it can be considered a self-attentive mechanism. The context vectors are essentially summaries of the words in the sentences. The matrix of context vectors $\boldsymbol{C} = [\boldsymbol{c}^{(1)}, \ldots, \boldsymbol{c}^{(n_S)}] \in \mathbb{R}^{d_v^{(S)} \times n_S}$ is constructed by grouping all the obtained context vectors together as columns. Finally, attention is calculated using $\boldsymbol{C}$ as feature input, producing the representation of the entire document in the context vector $\boldsymbol{c}^{(D)} \in \mathbb{R}^{d_v^{(D)}}$, where $d_v^{(D)}$ is the dimension of the value vectors in the attention model for document representation (Attention Module$_\text{D}$ in Fig. 8). This context vector can be used to classify the document, since it is essentially a summary of all the sentences (and therefore also the words) in the document.

Multi-level models can be used in a variety of tasks. For example, in [28], hierarchical attention is used in a recommender system to model user preferences at the long-term level and the short-term level. Similarly, [45] proposes a hierarchical model for recommending social media images based on user preferences. Hierarchical attention has also been successfully applied in other domains. For example, [46] proposes to use hierarchical attention in a video action recognition model to capture motion information at the the long-term level and the short-term level. Furthermore, [47] proposes a hierarchical attention model for cross-domain sentiment classification. In [48], a hierarchical attention model for chatbot response generation is proposed. Lastly, using image data, [49] proposes a hierarchical attention model for crowd counting.

3.1.3 Feature Representations

In a basic attention model, a single embedding or representation model is used to produce feature representations for the model to attend to. This is referred to as **single-representational attention**. Yet, one may also opt to incorporate multiple representations into the model. In [50], it is argued that allowing a model access to multiple

embeddings can allow one to create even higher quality representations. Similarly, [51] incorporates multiple representations of the same book (textual, syntactic, semantic, visual etc.) into the feature model. Feature representations are an important part of the attention model, but attention can also be an important part of the feature model. The idea is to create a new representation by taking a weighted average of multiple representations, where the weights are determined via attention. This technique is referred to as **multi-representational attention**, and allows one to create so-called meta-embeddings. Suppose one wants to create a meta-embedding for a word $\boldsymbol{x}$ for which E embeddings $\boldsymbol{x}^{(e_1)}, \ldots, \boldsymbol{x}^{(e_E)}$ are available. Each embedding $\boldsymbol{x}^{(e_i)}$ is of size d_{e_i}, for $i = 1, \ldots, E$. Since not all embeddings are of the same size, a transformation is performed to normalize the embedding dimensions. Using embedding-specific weight parameters, each embedding $\boldsymbol{x}^{(e_i)}$ is transformed into the size-normalized embedding $\boldsymbol{x}^{(t_i)} \in \mathbb{R}^{d_t}$, where d_t is the size of every transformed word embedding, as shown in (21).

$$\underset{d_t \times 1}{\boldsymbol{x}^{(t_i)}} = \underset{d_t \times d_{e_i}}{\boldsymbol{W}_{e_i}} \times \underset{d_{e_i} \times 1}{\boldsymbol{x}^{(e_i)}} + \underset{d_t \times 1}{\boldsymbol{b}_{e_i}}, \tag{21}$$

where $\boldsymbol{W}_{e_i} \in \mathbb{R}^{d_t \times d_{e_i}}$, and $\boldsymbol{b}_{e_i} \in \mathbb{R}^{d_t}$ are trainable, embedding-specific weights matrices. The final embedding $\boldsymbol{x}^{(e)} \in \mathbb{R}^{d_t}$ is a weighted average of the previously calculated transformed representations, as shown in (22).

$$\underset{d_t \times 1}{\boldsymbol{x}^{(e)}} = \sum_{i=1}^{E} \underset{1 \times 1}{a_i} \times \underset{d_t \times 1}{\boldsymbol{x}^{(t_i)}}. \tag{22}$$

The final representation $\boldsymbol{x}^{(e)}$ can be interpreted as the context vector from an attention model, meaning that the weights $a_1, \ldots, a_E \in \mathbb{R}^1$ are attention weights. Attention can be calculated as normally, where the columns of the features matrix $\boldsymbol{F}$ are the transformed representations $\boldsymbol{x}^{(t_1)}, \ldots, \boldsymbol{x}^{(t_E)}$. The query in this case can be ignored since it is constant in all cases. Essentially, the query is "Which representations are the most important?" in every situation. As such, this is a self-attentive mechanism.

While an interesting idea, applications of multi-representational attention are limited. One example of the application of this technique is found in [52], where a multi-representational attention mechanism has been applied to generate multi-lingual meta-embeddings. Another example is [53], where a multi-representational text classification model is proposed that incorporates different representations of the same text. For example, the proposed model uses embeddings from part-of-speech tagging, named entity recognizers, and character-level and word-level embeddings.

3.2 General Attention Mechanisms

This major category consists of attention mechanisms that can be applied in any type of attention model. The structure of this component can be broken down into the following sub-aspects: the attention score function, the attention alignment, and attention dimensionality.

3.2.1 Attention Scoring

The attention score function is a crucial component in how attention is calculated. Various approaches have been developed that each have their own advantages and disadvantages. An overview of these functions is provided in Table 2.

TABLE 2
Overview of score function (score$(\boldsymbol{q}, \boldsymbol{k}_l)$) forms.

Name	Function	Parameters
Additive (Concatenate) [3]	$\boldsymbol{w}^T \times \text{act}(\boldsymbol{W}_1 \times \boldsymbol{q} + \boldsymbol{W}_2 \times \boldsymbol{k}_l) + \boldsymbol{b})$	$\boldsymbol{w} \in \mathbb{R}^{d_w}$ $\boldsymbol{W}_1 \in \mathbb{R}^{d_w \times d_q}$ $\boldsymbol{W}_2 \in \mathbb{R}^{d_w \times d_k}$ $\boldsymbol{b} \in \mathbb{R}^{d_w}$
Multiplicative (Dot-Product) [4]	$\boldsymbol{q}^T \times \boldsymbol{k}_l$	-
Scaled Multiplicative [13]	$\frac{\boldsymbol{q}^T \times \boldsymbol{k}_l}{\sqrt{d_k}}$	-
General [4]	$\boldsymbol{k}_l^T \times \boldsymbol{W} \times \boldsymbol{q}$	$\boldsymbol{W} \in \mathbb{R}^{d_k \times d_q}$
Biased General [54]	$\boldsymbol{k}_l^T \times (\boldsymbol{W} \times \boldsymbol{q} + \boldsymbol{b})$	$\boldsymbol{W} \in \mathbb{R}^{d_k \times d_q}$ $\boldsymbol{b} \in \mathbb{R}^{d_k}$
Activated General [34]	$\text{act}(\boldsymbol{k}_l^T \times \boldsymbol{W} \times \boldsymbol{q} + b)$	$\boldsymbol{W} \in \mathbb{R}^{d_k \times d_q}$, $b \in \mathbb{R}^1$
Similarity [55]	similarity$(\boldsymbol{q}, \boldsymbol{k}_l)$	-

Each row of Table 2 presents a possible form for the function score$(\boldsymbol{q}, \boldsymbol{k}_l)$, as seen in (23), where $\boldsymbol{q}$ is the query vector, and $\boldsymbol{k}_l$ is the lth column of $\boldsymbol{K}$. Note that the score functions presented in this section can be more efficiently calculated in matrix form using $\boldsymbol{K}$ instead of each column separately. Nevertheless, the score functions are presented using $\boldsymbol{k}_l$ to more clearly illustrate the relation between a key and query.

$$\underset{1 \times 1}{e_l} = \text{score}(\underset{d_q \times 1}{\boldsymbol{q}}, \underset{d_k \times 1}{\boldsymbol{k}_l}). \tag{23}$$

Due to their simplicity, the most popular choices for the score function are the **concatenate score** function [3] and the **multiplicative score** function [4]. The multiplicative score function has the advantage of being computationally inexpensive due to highly optimized vector operations. However, the multiplicative function may produce non-optimal results when the dimension d_k is too large [56]. When d_k is large, the dot-product between $\boldsymbol{q}$ and $\boldsymbol{k}_l$ can grow large in magnitude. To illustrate this, in [13], an example is used where the elements of $\boldsymbol{q}$ and $\boldsymbol{k}_l$ are all normally distributed with a mean equal to zero, and a variance equal to one. Then, the dot-product of the vectors has a variance of d_k. A higher variance means a higher chance of numbers that are large in magnitude. When the softmax function of the alignment step is then applied using these large numbers, the gradient will become very small, meaning the model will have trouble converging [13]. To adjust for this, [13] proposes to scale the multiplicative function by the factor $\frac{1}{\sqrt{d_k}}$, producing the **scaled multiplicative score** function.

In [4], the multiplicative score function is extended by introducing a weights matrix $\boldsymbol{W}$. This form, referred to as the **general score** function, allows for an extra transformation of $\boldsymbol{k}_l$. The **biased general score** function [54] is a further extension of the general function that introduces a bias weight vector $\boldsymbol{b}$. A final extension on this function named the **activated general score** function is introduced in [34], and includes the use of both a bias weight b, and an activation function act().

The previously presented score functions are all based on determining a type of similarity between the key vector and the query vector. As such, more typical similarity measures, such as the Euclidean (L_2) distance and cosine similarity, can also be implemented [55]. These scoring methods are summarized under the **similarity score** function which is represented by the similarity() function.